Technical Documentation for Unified Experimental Pipeline on High-Dimensional Clinical Data

Bhavesh Sharad Yeole
Chethan Chinnabhandara Nagaraj
Sajith Kumar Santhosh
Sourav Salkoppalu Lingaraju

January 2026

Contents

1 Introduction

This document provides technical documentation for the software system developed for predictive modeling on high-dimensional clinical and biomedical datasets. The system integrates data simulation, feature extraction, embedding generation, model training, and performance evaluation into a unified and reproducible workflow.

The documentation is intended to support reproducibility, ease of use, and future extension of the developed framework.

2 System Requirements

The software system is implemented in Python and requires the following components:

- Python 3.8 or higher
- NumPy
- Pandas
- Scikit-learn
- XGBoost
- PyTorch
- Matplotlib

A Linux or Windows operating system with at least 8 GB RAM is recommended for large-scale experiments.

3 Project Structure

The project is organized into modular components, each responsible for a specific stage of the experimental workflow.

- `main copy.py` – Main execution script
- `simulator.py` – Data simulation module
- `toolbox.py` – Feature selection and embedding module
- `evaluation.py` – Evaluation and reporting module

This modular design supports independent testing and maintenance of individual components.

4 Installation and Setup

4.1 Environment Setup

Install the required dependencies using:

```
pip install numpy pandas scikit-learn xgboost torch matplotlib tabpfn
```

4.2 Project Setup

Clone or extract the project files into a working directory and ensure that all modules are accessible within the same environment.

5 Main Execution Module

5.1 Overview

The main execution module (`main copy.py`) serves as the entry point for running experiments. It coordinates data loading, simulation, model training, and evaluation.

5.2 Responsibilities

- Parse command-line arguments
- Load datasets
- Configure experiments
- Execute pipeline modules
- Collect results

5.3 Example Usage

```
python main.py --data dataset.csv --target label --method ALL --smogn
```

6 Simulation Module

6.1 Overview

The simulation module (`simulator.py`) introduces controlled imperfections into datasets by reducing sample size and applying oversampling techniques.

6.2 Main Functions

smogn_oversample() Implements SMOGN-style oversampling for regression by interpolating rare samples.

balance_dataset() Reduces dataset size and applies balancing techniques.

simulator_module() Provides a unified interface for simulation operations.

6.3 Functionality

- Rare sample identification
- Synthetic data generation
- Controlled sample reduction
- Reproducible sampling

7 Feature Selection and Embedding Module

7.1 Overview

The toolbox module (`toolbox.py`) implements DeepFS and GRACES-based feature selection and embedding generation methods.

7.2 Base Selector

The BaseSelector class provides a unified interface for feature selection methods, including `fit()` and `transform()` functions.

7.3 DeepFS Selector

DeepFS applies autoencoder-based dimensionality reduction and multivariate rank distance correlation to identify informative features in ultra high-dimensional data.

7.4 GRACES Selector

GRACES generates semantic embeddings using generative representation learning and contrastive policy optimization.

7.5 Toolbox Factory

The `get_toolbox()` function returns configured selector objects for experimental use.

8 Model Training Module

8.1 Overview

Model training is performed using embedding-based features and ensemble learning methods.

8.2 Supported Models

- XGBoost Classifier
- XGBoost Regressor
- TabPFN Classifier
- TabPFN Regressor

8.3 Training Procedure

- Data splitting
- Hyperparameter tuning
- Cross-validation
- Performance monitoring

9 Evaluation Module

9.1 Overview

The evaluation module (`evaluation.py`) computes task-specific performance metrics and generates summary reports.

9.2 Classification Metrics

- Accuracy
- Precision
- Recall
- F1-score
- Weighted F1-score

9.3 Regression Metrics

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- Coefficient of Determination (R^2)

9.4 Reporting

Results are formatted into tables and summaries for comparative analysis.

10 Workflow Execution

The experimental workflow follows the sequence:

$$\text{Preprocessing (External)} \rightarrow \text{Simulation} \rightarrow \text{Embedding} \rightarrow \text{XGBoost / TabPFN} \rightarrow \text{Evaluation} \rightarrow \text{Reporting}$$

This workflow is orchestrated through the main execution module.

11 Reproducibility

To ensure reproducibility, the system:

- Uses fixed random seeds
- Logs experimental parameters
- Stores configuration files
- Maintains consistent data splits

These practices enable exact replication of experimental results.

12 Troubleshooting

12.1 Common Issues

- Missing dependencies
- Incorrect file paths
- Memory limitations
- Incompatible Python versions

12.2 Recommended Solutions

- Verify package versions
- Check dataset format
- Reduce batch size
- Reinstall dependencies

13 Extensibility

The modular design allows easy extension through:

- New simulation methods
- Additional embedding models
- Custom evaluation metrics
- Alternative learning algorithms

Individual modules may be replaced without affecting the overall workflow.

14 Conclusion

This technical documentation describes the structure, implementation, and usage of the unified experimental pipeline for high-dimensional clinical data analysis. The framework supports reproducible experimentation, robust evaluation, and flexible extension for future research.